

Institut Supérieure d'Informatique et de
Multimédia de Sfax



Atelier de Programmation

Chapitre 2

Les bases de la programmation C

Enseignant : Ramzi ZOUARI

1^{ère} année License en Sciences de l'Informatique

Année universitaire : 2025-2026

Plan

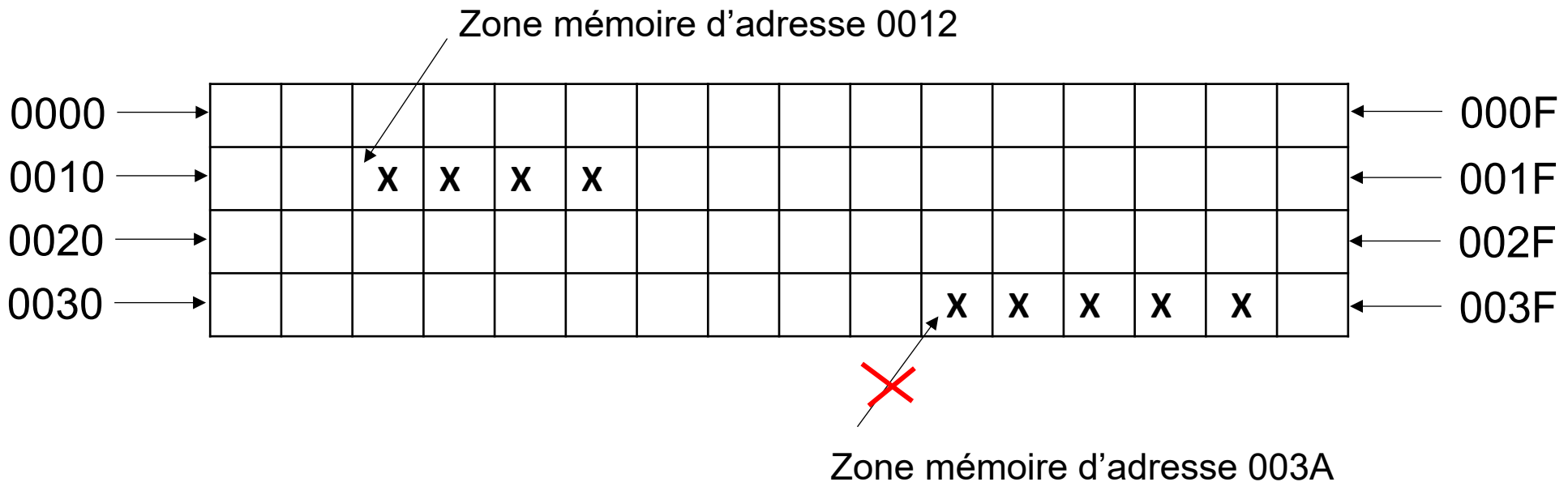
1. Les variables
2. Les types simples
3. Les constantes
4. Les opérateurs
5. Les entrées-sorties

1. Les variables

- On appelle variable une zone mémoire de l'ordinateur définie par:
 - Nom
 - Type
 - Contenu
 - Classe
- Dans une variable, on peut stocker une valeur et la modifier au cours du programme
- La classe d'une variable détermine:
 - L'adresse mémoire où elle est stockée
 - Sa durée de vie
 - Ses droits d'accès (modifier/supprimer)

1. Les variables

- La taille mémoire d'une variable dépend du type de la variable
- Lorsqu'une variable est supprimée, elle garde sa trace dans la mémoire mais le lien vers l'adresse mémoire de cette variable est supprimé



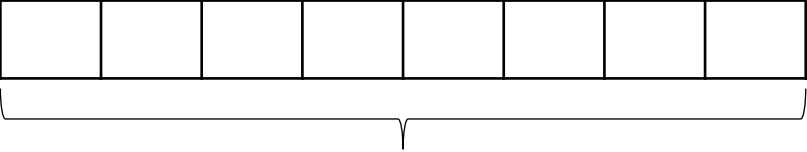
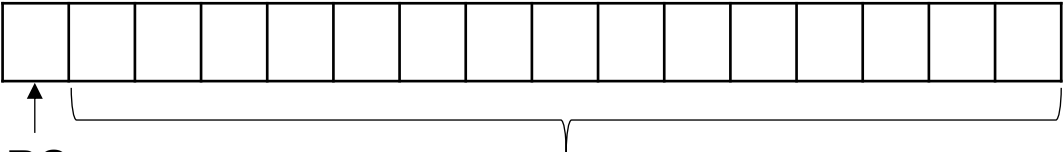
2. Les types de base

- Le langage C contient les types de base suivants:
 - Entier (**int**)
 - Réel simple (**float**)
 - Réel double précision (**double**)
 - Caractère (**char**)
- Les mots-clés **short** et **long** peuvent s'appliquer aux types **int** et **float**. Elles permettent d'influer sur la taille mémoire des entiers et des réels.
- Le mot-clé **unsigned** peut s'appliquer aux types **char** et **int** pour indiquer si le bit de poids fort doit être considéré ou non comme un bit de signe

2. Les types de base

Syntaxe	Type	Taille (octet)	Intervalle des valeurs
char <i>nom_variable</i> ;	Caractère	1	-128 ... +127
unsigned char <i>nom_variable</i> ;	Caractère non signée	1	0 ... 255
int <i>nom_variable</i> ;	Entier	4	-2 147 483 648 ... +2 147 483 647
long int <i>nom_variable</i> ; long <i>nom_variable</i> ;	Entier long		
unsigned long <i>nom_variable</i> ;	Entier long non signé	4	0 ... 4 294 967 295
unsigned int <i>nom_variable</i> ;	Entier non signé		
short int <i>nom_variable</i> ;	Entier short	2	-32 768 ... +32 767
float <i>nom_variable</i> ;	Réel simple précision	4	$-3.4e^{-38} \dots +3.4e^{+38} -1$
double <i>nom_variabe</i> ;	Réel double précision	8	$-1.7e^{-308} \dots +1.7e^{-308} -1$

2. Les types de base

Type	Taille (bits)	Représentation
char	8	 Bit de Signe (BS) Valeur positive entre 0 et 2^7
unsigned char	8	 Valeur positive entre 0 et 2^8
short int	16	 BS Valeur positive entre 0 et 2^{15}

2. Les types de base

Type	Taille (bits)	Représentation
int / long int / long	32	
unsigned int	32	
float	32	

2. Les types de base

Type	Taille (bits)	Représentation
double	64	63 62 52 51 50 49 1 0 ↑ BS Mantisse Exposant sur 11 bits Mantisse sur 52 bits

Remarques:

- Il existe le type ensemble vide: c'est le type **void**
- Les types **short int** et **long int** peuvent être abrégés en **short** et **long**
- Un caractère (**char**) peut être considéré comme un entier qu'on pourra utiliser dans une expression arithmétique

2. Les types de base

Type	Déclaration & Affectation	Affichage du contenu
int unsigned int long int short int	int a=5; unsigned int a=5; long int a=5; short int=5;	printf("a=%d", a) printf("a=%u", a) printf("a=%li", a) printf("a=%hi", a)
float	float a=5.5;	printf("a=%f", a)
double	double a=5.5;	printf("a=%lf", a)
char unsigned char	char a='C' unsigned char='C'	printf("a=%c", a) printf("a=%u", a)

Remarque:

- L'affichage d'une variable de type **unsigned char** avec le format **%u** permet de retourner le code ASCII (type entier) du caractère déclaré

2. Les types de base

- Les variables de type numérique (int-float-double) sont affichées en décimale (base 10)
- Pour afficher une variable numérique dans la base octale (base 8) ou hexadécimale (base 16), il suffit de changer le format d'affichage
 - Base octale: %o
 - Base Hexadécimale : %x

```
#include<stdio.h>
int main(){
    // Declaration et Affectation des variables
    int a=15;
    // Affichage de contenu des variables
    printf("\nDecimal: a=%d", a);
    printf("\nOctal: a=%o", a);
    printf("\nHexadecimal: a=%x", a);
    return 0;
}
```

```
Decimal: a=15
Octal: a=17
Hexadecimal: a=f
```

```
#include<stdio.h>
int main(){
    // Declaration et Affectation des variables
    char c='H';
    // Affichage de contenu des variables
    printf("c=%c", c);
    printf("\nLe code ASCII de c=%u", c);
    return 0;
}
```

```
c=H
c=72
```

3. Les constantes

- En C, une constante peut être déclarée en utilisant :
 - La directive **#define**
 - L'opérateur **const**
 - Le type **enum**
- Dans le cas de la directive **#define**, le compilateur ne réserve pas de l'espace mémoire pour la constante
- Dans le cas de l'opérateur **const**, le compilateur réserve de l'espace mémoire pour la constante mais on ne peut pas changer sa valeur

3. Les constantes

```
#include<stdio.h>
#define PI 3.14159;
int main(){
    int rayon=5;
    float perimetre;
    perimetre = 2*rayon*PI;
    printf("Perimetre=%f", perimetre);
    return 0;
}
```

```
#include<stdio.h>
int main(){
    const float PI=3.14159;
    int rayon=5;
    float perimetre;
    perimetre = 2*rayon*PI;
    printf("Perimetre=%f", perimetre);
    return 0;
}
```

3. Les constantes

- Les constantes de type énumération sont des types définissant un ensemble de constantes qui portent un nom appelé **enumérateur**
- Pour chaque énumérateur est associée une valeur entière de façon automatique en partant de la valeur 0 et avec un pas de 1

```
enum nom_enum {enumérateur_1, enumérateur_2, ..., enumérateur_n}
```

- **Exemple1:** **enum** couleurs {noir, bleu, vert, rouge}

noir	bleu	vert	Rouge
0	1	2	3

3. Les constantes

- **Exemple2:** enum couleurs {noir=-1, bleu, vert=3, rouge}

noir	bleu	vert	Rouge
-1	0	3	4

4. Les opérateurs

- Les opérateurs sont des symboles utilisés dans une expression pour relier entre les variables et les constantes
- Les principales classes d'opérateurs sont:
 - Les opérateurs arithmétiques
 - Les opérateurs logiques
 - Les opérateurs de manipulations de bits
 - Les opérateurs d'affectation
 - Les opérateurs d'adressage

4. Les opérateurs

Les opérateurs arithmétiques		
Symbole	Signification	Exemple
+	Addition	int a=6; a = a+5;
-	Soustraction	int a=6; a = a-5;
*	Multiplication	int a=6; a = a*5;
/	Division euclidienne	float a=6; a = a/5;
%	Reste de la division (Modulo)	int a=6; a = a%5;

4. Les opérateurs

Les opérateurs logiques		
Type	Symbole	Signification
Opérateurs relationnels	<	Inférieur
	<=	Inférieur ou égale
	>	Supérieur
	>=	Supérieur ou égale
	== !=	Égale à Différent de

- En C, le type booléen n'existe pas. Une expression logique retourne:
 - **1** si l'expression logique est vraie
 - **0** sinon
- Toute valeur **non nulle** est considérée comme vraie, et la valeur **nulle** comme fausse

4. Les opérateurs

Les opérateurs logiques		
Type	Symbole	Signification
Opérateurs relationnels	<	Inférieur
	<=	Inférieur ou égale
	>	Supérieur
	>=	Supérieur ou égale
	==	Égale à
	!=	Différent de

Exemple :

```
int a=7; float f=5.5; char c="w"
```

- `f >= 5 // vrai`
- `a != f // vrai`
- `c == "x" //faux`

4. Les opérateurs

Les opérateurs logiques		
Type	Symbole	Signification
Opérateurs de conjonction	&&	ET logique
		OU logique
	!	NOT logique

Exemple :

```
int a=7; float f=5.5; char c="w";
```

- `(a >= 6) && (c == "w" ) // vrai`
- `(a >= 6) || (c == "w" ) // vrai`
- `(f < 11) && (a > 100 ) // faux`

4. Les opérateurs

Les opérateurs de manipulation de bits		
Type	Symbole	Signification
Opérateurs arithmétiques « bit à bit »	~	NOR logique
	&	ET logique
		OU logique
	^	XOR logique

b1	b2	~b1	b1 & b2	b1 b2	b1 ^ b2
1	1	0	1	1	0
1	0	0	0	1	1
0	1	1	0	1	1
0	0	1	0	0	0

4. Les opérateurs

Les opérateurs de manipulation de bits					
b1	b2	~b1	b1 & b2	b1 b2	b1 ^ b2
1	1	0	1	1	0
1	0	0	0	1	1
0	1	1	0	1	1
0	0	1	0	0	0

Exemple :

```
int a, b, c;
```

```
a = 0x6DB7; // -0- | -0- | 0110 1101 | 1011 0111 |
```

```
b = 0xA726; // -0- | -0- | 1010 0111 | 0010 0110 |
```

```
c = a&b;    // -0- | -0- | 0010 0101 | 0010 0110 |
```

```
c = a|b;    // -0- | -0- | 1110 1111 | 1011 0111 |
```

```
c = a^b;    // -0- | -0- | 1100 1010 | 1001 0001 |
```

4. Les opérateurs

Les opérateurs de décalage		
Symbole	Signification	Exemple
>>	Décalage à droite	<pre>int a = 0x6db7; // 0110 1101 1011 0111 a = a >> 6; // 0000 0001 1011 0110 Le résultat est a = 0x01b6 = 0x1b6</pre>
<<	Décalage à gauche	<pre>int a = 0x6db7; // 0110 1101 1011 0111 a = a << 6; // 0110 1101 1011 0111 000000 // 0001 1011 0110 1101 1100 0000 Le résultat est a = 0x1b6dc0</pre>

- Dans le cas du décalage à droite >>, les bits à droite sont perdus. Les positions binaires rendus vacantes seront remplacés par des 0
- Dans le cas du décalage à gauche <<, tous les bits se décalent à gauche. Les bits à droite vacants seront remplacés par des 0
- Les opérateurs de décalage >> et << peuvent être appliqués sur des entiers au format décimale

4. Les opérateurs

Les opérateurs d'affectation		
Type	Symbole	Exemple
Affectation simple	=	float a, b, c; a = b + 2*b/c;
Affectation composée	+= *= -= /= $\text{\&=}$ = <<= >>=	$a \text{ += } 5 \Leftrightarrow a = a + 5$ $a \text{ *= } b + c \Leftrightarrow a = a * (b + c)$ $a \text{ \&= } 0x50 \Leftrightarrow a = a \& 0x50$ $a \text{ <<= } 5 \Leftrightarrow a = a \text{ << } 5$
Incrémentation et décrémentation	++ ou --	int i=99, j=2, k; $k = 5 + i \text{++}$ // k=5+i puis i=i+1 $k = 5 + \text{++}i$ // i=i+1 puis k=5+i $K=5 + \text{--}j$ // j=j-1 puis k=5+j

4. Les opérateurs

- L'opérateur d'adressage & permet renvoyer l'adresse d'une variable stockée en mémoire

```
#include<stdio.h>
int main() {
    int a = 50;
    printf("Adresse de la variable a est %p", &a);
    return 0;
}
```

- Pour afficher le contenu d'une adresse, il faut utiliser un pointeur vers cette adresse

```
#include<stdio.h>
int main() {
    int a = 50;
    int *p = &a;
    printf("Adresse %p contient la valeur %d", &a, *p);
    return 0;
}
```

5. Les entrée/sorties

- Pour transmettre des données saisies au clavier à un programme ou pour faire afficher des données, il faut des commandes adéquates
- En C, la bibliothèque **stdio.h** contient les fonctions d'entrée/sortie:
 - `printf()` : permet l'affichage formaté des données sur le terminal
 - `scanf()`: permet la saisie formatée des données à partir du clavier
- Syntaxe:
 - **`printf("format_affichage", [argument_1, argument_2, ..., argument_n]);`**
 - **`scanf("format_saisie", [argument_1, argument_2, ..., argument_n]);`**

5. Les entrée/sorties

Les formats d'affichage (printf)		
Format	Type	Exemple
%d	nombre décimal	int a = 5; printf("a=", %d); // affichage : a=5 printf("a=", %04d); // affichage : a=0005
%u	Entier non signé	unsigned int a = 5; printf("a=", %u); // affichage : a=5 printf("a=", %04u); // affichage : a=0005
%x , %X	Entier en Hexadécimal	int a= 7194; printf("a=", %x); // affichage : a=1c1a printf("a=", %X); // affichage : a=1C1A
%o	Entier en octale	int a = 7194; printf("a=", %o); // affichage : a=16032
%c	Caractère ASCII	char a = 'C'; printf("a=", %c); // affichage : a=C

5. Les entrée/sorties

Les formats d'affichage (printf)		
Format	Type	Exemple
%s	Chaine de caractère	char [] a = "langage C"; printf("a=", %s); // affichage : a=langage C
%f ou %lf	Nombre à virgule flottante	float a = 150.2550; printf("a=", %f); // affichage : a=150.2550 printf("a=", %.0f); // affichage : a=150 printf("a=", %.2f); // affichage : a=150.26
%e	Nombre à virgule flottante en format exponentiel	int a= 71.95; printf("a=", %e); // affichage : a=7.195000e+01 printf("a=", %.2e); // affichage : a=7.19e+01
%p	Pointeur (adresse mémoire d'une variable)	int a = 7194; printf("@a=%p", a); // affichage : @a=a771dffcdc

5. Les entrée/sorties

Les séquences d'échappement dans l'affichage		
Caractère	Rôle	Exemple
\n	Saut de ligne	<pre>printf("Langage \nC\n"); // affichage :Langage C</pre>
\t	Tabulation	<pre>printf("Langage \tC\n"); // affichage :Langage C</pre>

5. Les entrée/sorties

Les formats de saisie (scanf)		
Format	Type	Exemple
%d %ld %hd	entier décimal de type int entier décimal de type long int entier décimal de type short int	int a; printf("Donner a:\n"); scanf("%d" , &a); printf("a=%d", a);
%u %lu %hu	entier décimal de type int non signé entier décimal de type long int entier décimal de type short int	int a; printf("Donner a:\n"); scanf("%d" , &a); printf("a=%u", a);
%f %lf	Nombre à virgule flottante de type float Nombre à virgule flottante de type double	float a; printf("Donner a:\n"); scanf("%f", &a);
%o %ho %lo	Entier octale de type int Entier octale de type short Entier octale de type long	int a; printf("Donner a:\n"); scanf("%o", &a);

5. Les entrée/sorties

Les formats de saisie (scanf)		
Format	Type	Exemple
%x %hx %lx	Entier hexadécimale de type int Entier hexadécimale de type short Entier hexadécimale de type long	int a; printf("Donner a:\n"); scanf("%x", &a);
%c	Caractère ASCII	char a; printf("Donner a:\n"); scanf("%c", &a);
%s	Chaine de caractères	char [] a; printf("Donner a:\n"); scanf("%s", &a);